

L4c: Shortest-Path Algorithms

CHEME 5800 Cornell University

Objectives for Today

Today we find the least-cost routes from one starting vertex and compare how Dijkstra and Bellman-Ford calculate them.

Today's concepts

- **Shortest paths and relaxation**: calculate path cost and update distance and predecessor maps.
- **Dijkstra's algorithm**: explain why nonnegative edge weights allow a selected distance to be declared final.
- **Bellman-Ford**: explain repeated relaxation, negative edges, and detection of negative cycles reachable from the source.

Lecture: [Shortest-Path Algorithms](#)

Lecture and Lab

Lecture: Shortest-Path Algorithms

Define route cost, follow both algorithms step by step, and distinguish a negative edge that lowers cost from a negative cycle that can be repeated to keep lowering it.

Lab L4d: Production Planning with Shortest Paths

Predict a least-cost production route, compute it with Dijkstra, verify it with Bellman-Ford, and determine how large a discount must be to change the least-cost route.

Which Route Has the Lowest Total Cost?

An edge weight can represent travel time, communication delay, monetary cost, or the number of unit operations in a chemical process.

The common question is:

Among all feasible routes from a source to a target,
which route has the **smallest total edge weight**?

Checking every route separately is usually impractical: even a graph with no cycles can have exponentially many routes.

A Feasible Route Follows Directed Edges

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a finite directed graph with vertex set $\mathcal{V}$ and edge set $\mathcal{E}$. The function $w : \mathcal{E} \rightarrow \mathbb{R}$ assigns each edge a real-valued cost.

We write $|\mathcal{V}|$ for the number of vertices and $|\mathcal{E}|$ for the number of edges. A route P from source s to target t with k edges has the ordered vertices:

$$P = \langle v_0, v_1, \dots, v_k \rangle, \quad v_0 = s, \quad v_k = t,$$

with $(v_i, v_{i+1}) \in \mathcal{E}$ for $i = 0, \dots, k-1$.

Its total weight is the sum of its edge weights:

$$\underbrace{w(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})}_{\text{route cost}}.$$

Shortest-Path Distance Measures Minimum Cost

Let $\mathcal{P}_{s,t}$ be the set of feasible routes from s to t . A negative-weight cycle is a directed cycle whose edge weights sum to a negative number. If no such cycle lies on an s -to- t route, define:

$$d(s, t) = \begin{cases} \min_{P \in \mathcal{P}_{s,t}} w(P), & \mathcal{P}_{s,t} \neq \emptyset, \\ +\infty, & \mathcal{P}_{s,t} = \emptyset. \end{cases}$$

An unreachable target has $d(s, t) = +\infty$. For a finite minimum, $\min$ gives the cost and $\arg \min$ gives the routes with that cost. Choose one such route and call it P^* :

$$P^* \in \arg \min_{P \in \mathcal{P}_{s,t}} w(P), \quad w(P^*) = d(s, t).$$

Record Distances and Predecessors

After finding shortest paths from source s , we record two values for each vertex $v \in \mathcal{V}$:

$$\underbrace{\text{dist}[v] = d(s, v)}_{\text{minimum route cost}}, \quad \underbrace{\text{prev}[v]}_{\text{previous vertex on the chosen route}}.$$

- **nothing** means no predecessor is recorded; the source has $\text{prev}[s] = \text{nothing}$.
- Follow predecessors backward from a target, then reverse the sequence, to reconstruct one minimum-cost route.
- If several routes tie, the predecessor map may record any one of them.
- An unreachable vertex keeps distance $+\infty$ and no predecessor.

A Shortest Path Contains Shortest Paths

Suppose a shortest path from source s to target t passes through vertex u .

The part of this path from s to u must also be a shortest path from s to u .

If we could reach u more cheaply, we could take that cheaper route and then follow the original path from u to t . This would lower the total cost to t , contradicting our assumption that the original path was shortest.

This property is called **optimal substructure**.

Why this helps: extend the cheapest route found to u to look for cheaper routes to its neighbors.

Relaxation Starts with One Known Route

At initialization, the zero-edge route from s to itself is the only known route:

$$\text{dist}[v] = \begin{cases} 0, & v = s, \\ +\infty, & v \neq s, \end{cases} \quad \text{prev}[v] = \text{nothing}.$$

A finite estimate $\text{dist}[v]$ is always the weight of an actual source-to- v route discovered so far.

During the calculation, $\text{dist}[v]$ is the lowest cost found so far. It can decrease when we find a cheaper route.

Relaxation Checks for a Cheaper Route

Suppose we know a route to u . Following edge $(u, v) \in \mathcal{E}$ extends that route to v . We use $\text{alt}(u, v)$, short for **alternative route cost**, for the cost of reaching v this way:

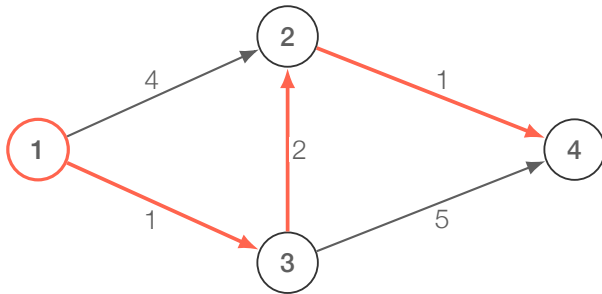
$$\underbrace{\text{alt}(u, v) = \text{dist}[u] + w(u, v)}_{\text{candidate route cost to } v}.$$

Relaxation updates both maps only when the candidate is strictly cheaper:

$$(\text{dist}[v], \text{prev}[v]) \leftarrow \begin{cases} (\text{alt}(u, v), u), & \text{alt}(u, v) < \text{dist}[v], \\ (\text{dist}[v], \text{prev}[v]), & \text{alt}(u, v) \geq \text{dist}[v]. \end{cases}$$

In the algorithm, alt stores this cost for the current edge. Tied costs leave the predecessor unchanged.

Example: A Four-Vertex Graph



Edge labels give costs. Red edges mark the minimum-cost route from source 1 to target 4. We first examine how relaxation improves the estimate for vertex 2.

Example: Relaxing One Edge

From source 1, the direct edges give $\text{dist}[2] = 4$ and $\text{dist}[3] = 1$. Both vertices initially have predecessor 1.

Can we reach vertex 2 more cheaply through vertex 3? For edge $(3, 2)$, the alternative route cost is:

$$\text{alt}(3, 2) = \text{dist}[3] + w(3, 2) = 1 + 2 = 3.$$

Since $3 < 4$, we update both maps:

$$\text{dist}[2] \leftarrow 3, \quad \text{prev}[2] \leftarrow 3.$$

Following predecessors from vertex 2 leads to vertex 3 and then vertex 1. Reversing this sequence gives the cheaper route $1 \rightarrow 3 \rightarrow 2$. We still need to establish when a distance estimate is final.

What Does Relaxation Guarantee?

Any route we discover costs at least as much as the shortest route. For a vertex v with a finite shortest-path distance:

$$\underbrace{d(s, v) \leq \text{dist}[v]}_{\text{estimate is at least the minimum}} .$$

Immediately after relaxing (u, v) , with $u \neq v$, the estimate for v is no greater than the cost of reaching it through u :

$$\underbrace{\text{dist}[v] \leq \text{dist}[u] + w(u, v)}_{\text{after checking the route through } u} .$$

If $\text{dist}[u]$ later decreases, we may need to check (u, v) again.

Dijkstra and Bellman-Ford use the same relaxation rule.
Dijkstra selects a vertex and checks its outgoing edges.
Bellman-Ford checks every edge repeatedly.

Dijkstra Requires Nonnegative Edge Weights

Dijkstra selects the smallest distance estimate among vertices not yet selected. For that estimate to be final, every edge must satisfy:

$$w(u, v) \geq 0 \quad \text{for every } (u, v) \in \mathcal{E}.$$

Initialize:

- the distance and predecessor maps defined earlier;
- an empty set $\mathcal{S}$ of selected vertices;
- a min-priority queue $\mathcal{Q}$, using distance estimates as priorities.

Start with only s in $\mathcal{Q}$, at priority zero. The queue returns the vertex with the smallest estimate first.

Dijkstra Finds One Shortest Distance at a Time

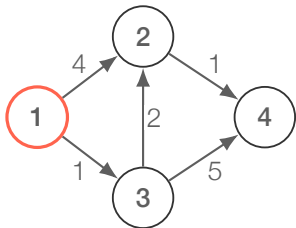
While the min-priority queue $\mathcal{Q}$ is not empty:

1. Remove the vertex u with the smallest distance estimate from $\mathcal{Q}$.
2. If $u \in \mathcal{S}$, skip it; otherwise add u to $\mathcal{S}$. Its distance is now final.
3. For every outgoing edge (u, v) , compute $\text{alt} = \text{dist}[u] + w(u, v)$.
4. If $\text{alt} < \text{dist}[v]$, update the distance and set $\text{prev}[v] = u$. Insert v into $\mathcal{Q}$, or lower its priority to alt if it is already queued.

The adjacency list gives the outgoing edges and their weights.

Dijkstra Trace on the Worked Graph

Each row follows relaxation from the selected vertex. Coral marks changed estimates; the source distance stays zero.



Source 1; edge labels give costs.

Selected	dist[2]	dist[3]	dist[4]	Queue
----------	---------	---------	---------	-------

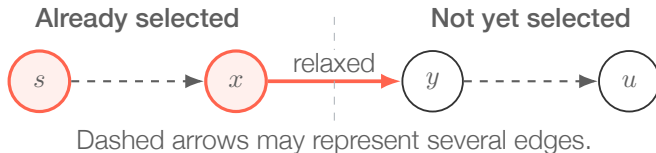
Initial	∞	∞	∞	1:0
1	4	1	∞	3:1, 2:4
3	3	1	6	2:3, 4:6
2	3	1	4	4:4
4	3	1	4	Empty

Queue entries are `vertex:priority`, in increasing priority order.

The predecessors are $\text{prev}[4] = 2$, $\text{prev}[2] = 3$, and $\text{prev}[3] = 1$. Reverse the backward sequence to recover $1 \rightarrow 3 \rightarrow 2 \rightarrow 4$.

Why Can't a Selected Distance Improve Later?

The source's shortest-path distance is zero. Assume all previously selected distances are correct. Suppose a route from s to the next vertex u costs less than $\text{dist}[u]$.



Let y be the first vertex on this route not yet selected, and x its predecessor. Relaxing (x, y) gave y an estimate no greater than the cost of reaching y along this route. The remaining edges add no negative cost, so:

$$\text{dist}[y] \leq \text{cost of the supposed cheaper route} < \text{dist}[u].$$

This contradicts our choice of u . Hence $\text{dist}[u] = d(s, u)$.

Negative Edges Can Change a Selected Distance

Consider a different graph with source vertex 1 and three directed edges:

$$w(1, 2) = 2, \quad w(1, 3) = 5, \quad w(3, 2) = -4.$$

If we apply Dijkstra's selection rule despite the negative edge, vertex 2 is selected with distance 2 before vertex 3.

Yet the route through vertex 3 has lower cost:

$$1 \rightarrow 3 \rightarrow 2, \quad 5 - 4 = 1.$$

Thus, a selected distance need not be final. Our implementation rejects negative-edge inputs before applying the selection rule.

Dijkstra: Computational Cost

With a weighted adjacency list and a binary-heap min-priority queue, Dijkstra runs in:

$$\underbrace{\mathcal{O}((|\mathcal{V}| + |\mathcal{E}|) \log |\mathcal{V}|)}_{\text{Dijkstra running time}}.$$

- Each reachable vertex is selected once.
- Each outgoing edge is examined when its starting vertex is selected.
- Queue insertions, removals, and priority updates each take $\mathcal{O}(\log |\mathcal{V}|)$ time.

Scope: the bound matches the course implementation using adjacency lists and a binary-heap priority queue.

Bellman-Ford Repeatedly Relaxes Every Edge

Bellman-Ford checks every edge during each **pass**. Repeating these checks allows later discoveries to lower earlier distance estimates, including when some weights are negative.

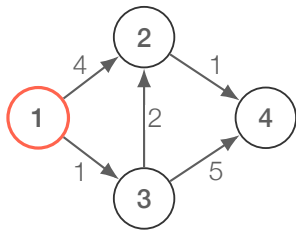
Initialize the same distance and predecessor maps. Then repeat at most $|\mathcal{V}| - 1$ times:

1. Visit every edge $(u, v) \in \mathcal{E}$.
2. If $\text{dist}[u]$ is finite and the candidate is cheaper, relax (u, v) .
3. If no distance changes during a complete pass, stop.

Each update is available to edges checked later in the same pass. Thus, edge order can change how many passes are needed.

Bellman-Ford: Following One Pass

Starting at source 1, visit the edges in the listed order. Coral marks an estimate changed by that relaxation.



$\text{dist}[1] = 0$ throughout.

Edge relaxed	$\text{dist}[2]$	$\text{dist}[3]$	$\text{dist}[4]$
Initial	∞	∞	∞
(1, 2)	4	∞	∞
(1, 3)	4	1	∞
(3, 2)	3	1	∞
(2, 4)	3	1	4
(3, 4)	3	1	4

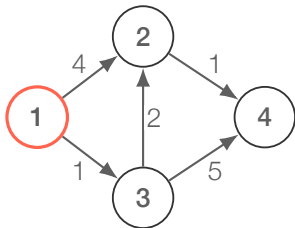
Relaxing (3, 2) lowers the estimate for vertex 2 from 4 to 3. The next edge, (2, 4), uses that improvement immediately to give $3 + 1 = 4$.

Bellman-Ford: Comparing Edge Orders

Reverse the preceding edge order:

$(3, 4), (2, 4), (3, 2), (1, 3), (1, 2).$

Each triple gives $(\text{dist}[2], \text{dist}[3], \text{dist}[4])$. Coral marks changes during that pass.



Pass	Listed order	Reversed order
0	(∞, ∞, ∞)	(∞, ∞, ∞)
1	$(\text{3}, 1, 4)$	$(\text{4}, 1, \infty)$
2	$(3, 1, 4)$; stop	$(\text{3}, 1, \text{5})$
3	Stopped	$(3, 1, \text{4})$

The listed order stops after pass 2 makes no changes. Reversing the order puts $(2, 4)$ before $(3, 2)$, so vertex 4 improves one pass later.

Why $|\mathcal{V}| - 1$ Passes Are Enough

Assume no negative-weight cycle is reachable from the source. After k passes, distances are correct for every vertex with a shortest path containing at most k edges.

If a route from the source repeats a vertex, the intervening edges form a cycle. Removing that cycle cannot increase the cost because its total weight is nonnegative.

We can therefore choose a shortest path with no repeated vertices, called a **simple path**. It visits at most $|\mathcal{V}|$ vertices and contains at most:

$$\underbrace{|\mathcal{V}| - 1}_{\text{maximum edges in a simple path}}$$

This is why $|\mathcal{V}| - 1$ relaxation passes are sufficient.

Detecting a Reachable Negative Cycle

After the relaxation passes, check every edge once more.

If an edge from a reachable vertex can still lower a distance estimate, there is a reachable cycle C with total weight $w(C) < 0$. For a route P through C , repeating the cycle gives costs:

$$w(P), \quad w(P) + w(C), \quad w(P) + 2w(C), \quad \dots \longrightarrow -\infty \quad \text{when } w(C) < 0.$$

For any target reachable from that cycle, repeating the cycle keeps making the route cheaper. There is no minimum cost.

A negative cycle unreachable from s is skipped because its vertices keep distance $+\infty$.

Bellman-Ford: Computational Cost

In the worst case, Bellman-Ford scans every edge during each of $|\mathcal{V}| - 1$ passes and during the cycle check:

$$\underbrace{\mathcal{O}(|\mathcal{V}| |\mathcal{E}|)}_{\text{running time}}, \quad \underbrace{\mathcal{O}(|\mathcal{V}|)}_{\text{distance and predecessor storage}}.$$

We can stop before the maximum number of passes if a complete pass changes no distances.

Bellman-Ford can find shortest paths with negative edges and report when a reachable negative cycle prevents a minimum cost.

Choosing Between Dijkstra and Bellman-Ford

Question	Dijkstra	Bellman-Ford
Weights	All nonnegative	May be negative
Update rule	Select smallest estimate; relax outgoing edges	Relax every edge repeatedly
Negative cycles	Not supported	Detects reachable negative cycles
Worst-case time	$\mathcal{O}((\mathcal{V} + \mathcal{E}) \log \mathcal{V})$	$\mathcal{O}(\mathcal{V} \mathcal{E})$
Storage used here	Weighted adjacency list	Edge list

Our Bellman-Ford implementation reports an error for a reachable negative cycle.

Both Algorithms Find the Same Minimum Costs

On our four-vertex graph, both algorithms give the same shortest-path distances:

$$\text{dist}_{\text{Dijkstra}}[v] = \text{dist}_{\text{Bellman-Ford}}[v] \quad \text{for every vertex } v.$$

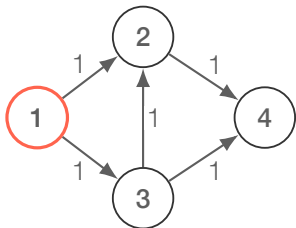
Both predecessor maps reconstruct the unique minimum-cost route to vertex 4:

$$P^* = \langle 1, 3, 2, 4 \rangle, \quad w(P^*) = 1 + 2 + 1 = 4.$$

All weights are nonnegative, so both algorithms apply. They use different orders and numbers of edge checks to find the same minimum costs.

Breadth-First Search When Every Edge Costs 1

If every edge costs 1, a route's cost equals its number of edges. Breadth-first search, used in L4b, finds routes with the fewest edges.

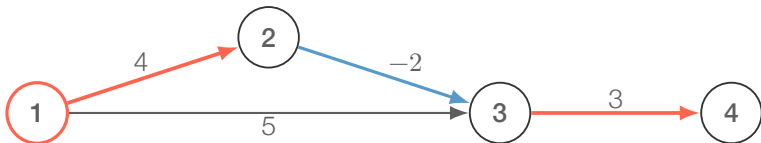


Same connections; every weight is now 1.

Route from 1 to 4	Cost
$1 \rightarrow 2 \rightarrow 4$	2
$1 \rightarrow 3 \rightarrow 4$	2
$1 \rightarrow 3 \rightarrow 2 \rightarrow 4$	3

The two-edge routes now tie for the minimum cost of 2. With the original weights, the three-edge route cost 4 and was cheapest.

A Shortest Path Can Include a Negative Edge

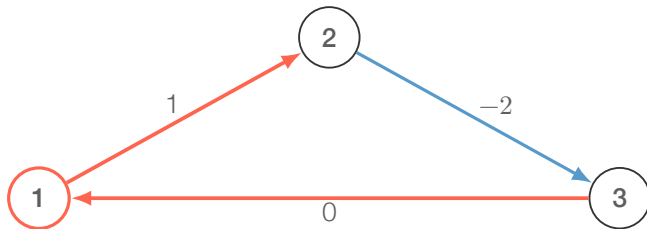


Bellman-Ford finds the minimum-cost route:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4, \quad 4 - 2 + 3 = 5.$$

This graph has no cycles, so the negative edge cannot be repeated to keep lowering the cost.

A Negative Cycle Keeps Lowering Cost



The cycle weight is:

$$w(C) = 1 - 2 + 0 = -1.$$

Each trip around the cycle lowers the route cost by one. Bellman-Ford's extra check finds another possible improvement and reports an error: there is no minimum-cost route to any vertex in this graph.

Check the Routes, Costs, and Expected Errors

The notebook checks the calculated results and the expected errors:

- **Nonnegative graph:** both algorithms return $\langle 1, 3, 2, 4 \rangle$ with target cost 4 and identical distance maps.
- **Negative edge, no negative cycle:** Bellman-Ford returns $\langle 1, 2, 3, 4 \rangle$ with target cost 5.
- **Expected errors:** Dijkstra raises an error for the negative edge; Bellman-Ford raises an error for the reachable negative cycle.

What we check: all distances on the nonnegative graph, selected routes and costs, and expected errors.

Lab L4d Applies Both Algorithms

The lab represents production steps and their costs as a weighted directed graph.

1. Predict the least-cost route by inspecting the process costs.
2. Compute the route with Dijkstra because all costs are nonnegative.
3. Verify the distance and route with Bellman-Ford.
4. Discount one production step and find the step cost at which the two routes have equal total cost.

We then examine which route is cheaper above and below that step cost.

Summary

We compared two algorithms for finding minimum-cost routes.

Key takeaways

- **Route costs and relaxation:** We summed edge weights, improved distance estimates by relaxation, and used predecessors to reconstruct routes.
- **Dijkstra's algorithm:** We used a priority queue to select the smallest estimate and showed why nonnegative weights make it final.
- **Bellman-Ford's algorithm:** We used repeated relaxation to handle negative edges and an extra scan to detect reachable negative cycles.

In [the L4d lab](#), we apply these methods to production costs and equipment discounts.

That's a wrap. Which property of the edge weights tells you which shortest-path algorithm is valid?